api-adapter

API Adapter

The API adapter provides an interface for REST API and WebSocket testing. It abstracts HTTP communication, authentication, and WebSocket connections behind a common contract.

APIAdapter Interface

Defined in `adapter_api.go`:

```
type APIAdapter interface {
    Login(ctx context.Context, credentials Credentials) (string,
        error)
    LoginWithRetry(ctx context.Context, credentials Credentials,
        retries int) (string, error)
    Get(ctx context.Context, path string) (int,
        map[string]interface{}, error)
    GetRaw(ctx context.Context, path string) (int, []byte, error)
    GetArray(ctx context.Context, path string) (int,
        []interface{}, error)
    PostJSON(ctx context.Context, path, body string) (int,
        []byte, error)
    PutJSON(ctx context.Context, path, body string) (int, []byte,
        error)
    Delete(ctx context.Context, path string) (int, []byte, error)
    WebSocketConnect(ctx context.Context, path string)
        (WebSocketConn, error)
    SetToken(token string)
    Available(ctx context.Context) bool
}
```

Method Summary

Method	Returns	Purpose
`Login`	`(token, error)`	Authenticate and get a JWT token
`LoginWithRetry`	`(token, error)`	Login with exponential backoff

Method	Returns	Purpose
`Get`	`(statusCode, jsonObject, error)`	GET with parsed JSON object response
`GetRaw`	`(statusCode, body, error)`	GET with raw byte response
`GetArray`	`(statusCode, jsonArray, error)`	GET with parsed JSON array response
`PostJSON`	`(statusCode, body, error)`	POST with JSON body
`PutJSON`	`(statusCode, body, error)`	PUT with JSON body
`Delete`	`(statusCode, body, error)`	DELETE request
`WebSocketConnect`	`(WebSocketConn, error)`	Establish a WebSocket connection
`SetToken`	`--`	Set the JWT token for authenticated requests
`Available`	`bool`	Check if the API server is reachable

Supporting Types

Credentials

```go
type Credentials struct {
    Username string `json:"username"`
    Password string `json:"password"`
    URL      string `json:"url"`
}
```

WebSocketConn

```go
type WebSocketConn interface {
    WriteMessage(data []byte) error
    ReadMessage() ([]byte, error)
    Close() error
}
```

HTTPAPIAdapter

The built-in implementation wraps the `pkg/httpclient.APIClient` for REST operations and uses `gorilla/websocket` for WebSocket connections.

Constructor

```
adapter := userflow.NewHTTPAPIAdapter(
    "http://localhost:8080",
    // optional httpclient.ClientOption values
)
```

The base URL is used for all requests. It accepts optional `httpclient.ClientOption` values that configure the underlying HTTP client (timeouts, TLS settings, etc.).

Authentication

`Login()` delegates to `httpclient.APIClient.Login()`, which POSTs credentials to the server's login endpoint and stores the returned JWT token internally. Subsequent requests automatically include the `Authorization: Bearer <token>` header.

`LoginWithRetry()` adds exponential backoff, retrying up to the specified number of times. This is useful when the server may still be starting up.

`SetToken()` allows setting a pre-obtained token directly without calling the login endpoint.

WebSocket Support

`WebSocketConnect()` converts the base URL scheme from `http://` to `ws://` (or `https://` to `wss://`), appends the path, and dials with a 10-second handshake timeout. If a token is set, it is included as a `Bearer` authorization header.

The returned `WebSocketConn` wraps a `gorilla/websocket.Conn`:

```
conn, err := adapter.WebSocketConnect(ctx, "/api/v1/ws")
if err != nil {
    return err
}
defer conn.Close()

err = conn.WriteMessage([]byte(`{"type":"subscribe","channel":"events"}
        `))
msg, err := conn.ReadMessage()
```

Availability

`Available()` performs a `GET /health` request and returns true if the status code is below 500.

Example: API Health Challenge

The simplest API challenge -- checks a single endpoint:

```
adapter := userflow.NewHTTPAPIAdapter("http://localhost:8080")

challenge := userflow.NewAPIHealthChallenge(
    "CH-API-001",
    adapter,
    "/api/v1/health",
    200,
    nil,
)
```

This GETs `/api/v1/health` and asserts the response status code equals 200.

Example: Multi-Step API Flow

```
flow := userflow.APIFlow{
    Name: "crud-resources",
    Credentials: userflow.Credentials{
        Username: "admin",
        Password: "password",
    },
    Steps: []userflow.APIStep{
        {
            Name:           "create-resource",
            Method:         "POST",
            Path:           "/api/v1/resources",
            Body:           `{"name":"test-
     resource","type":"document"}`,
            ExpectedStatus: 201,
            ExtractTo: map[string]string{
                "id": "resource_id",
            },
        },
        {
            Name:           "get-resource",
            Method:         "GET",
            Path:           "/api/v1/resources/{{resource_id}}",
            ExpectedStatus: 200,
            Assertions: []userflow.StepAssertion{
                {
```

```
                    Type:    "response_contains",
                    Target:  "body",
                    Value:   "test-resource",
                    Message:
        "response should contain the resource name",
                },
            },
        },
        {
            Name:           "delete-resource",
            Method:         "DELETE",
            Path:           "/api/v1/resources/{{resource_id}}",
            ExpectedStatus: 200,
        },
    },
}

challenge := userflow.NewAPIFlowChallenge(
    "CH-API-002",
    "CRUD Resources",
    "Create, read, and delete a resource",
    []challenge.ID{"CH-API-001"},
    adapter,
    flow,
)
```

Variable Extraction

The `ExtractTo` field on `APIStep` maps JSON response fields to variable names. Variables are substituted in subsequent steps using `{{variable_name}}` placeholders in both `Path` and `Body` fields.

In the example above, the `create-resource` step extracts the `id` field from the response JSON and stores it as `resource_id`. The `get-resource` step then uses `{{resource_id}}` in its path.

Step Assertion Types

The `evaluateStepAssertion` function supports:

Type	Value Type	Pass Condition
`status_code`	`int` or `float64`	HTTP status equals value
`response_contains`	`string`	Response body contains value
`not_empty`	--	Response body has non-zero length